Mentorship FGA — Authorization Model

Fine-grained authorization model for the LFX Mentorship service. Prototyped and validated here before being integrated into the LFX Platform v2 production stack.

Model overview

Four types, each scoped to its parent via a relation:

```
project:<slug>
  └── program:<id>
        └── application:<id>
              └── task:<id>
```

Type	Key relations	Key permissions
`project`	`super_admin`, `program_approver`, `program_admin`	`can_create_program`, `can_approve_program`
`program`	`program_admin`, `mentor` (+ inherited from `project`)	`can_view`, `can_invite_mentor`, `can_create_term`, `can_add_prerequisite_task`, `can_add_task`, `can_view_applications`, `can_decide_application`, `can_approve`
`application`	`mentee` (+ inherited from `program`)	`can_view`, `can_decide`, `can_add_task`, `can_withdraw`, `can_reapply`
`task`	`assignee` (+ inherited from `application`)	`can_view`, `can_update_status`

> **Review note:** tasks may only be added by a mentor while the application is in the `Accepted` state. FGA has no concept of application lifecycle state, so this check is enforced in the mentorship service's business logic, layered on top of `can_add_task` — not modeled as an FGA relation.

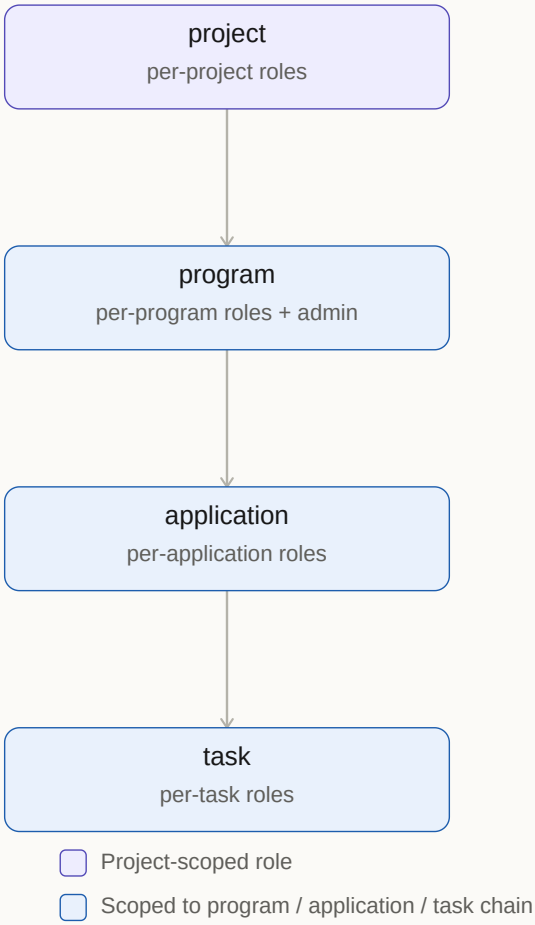

Figure 1 — Object hierarchy. Each project is its own authorization boundary; permissions cascade top to bottom via `X from Y` projections.

Role summary

Role	Scope	What they can do
`super_admin`	per-project	everything within that project
`program_approver`	per-project	view + approve programs in their project, view applications — deliberately excluded from admin so a program cannot approve itself
`program_admin`	per-project + per-program	create programs (project-level); full control of their specific program and its applications/tasks, including deciding applications
`mentor`	per-program	view program + applications, create terms, add prerequisite and non-prerequisite tasks, update task status — cannot invite mentors and **cannot** accept/reject applications
`mentee`	per-application	view, withdraw, and re-apply to their own application and tasks; withdraw/re-apply require ownership — the application ID alone is not a credential

Process flows

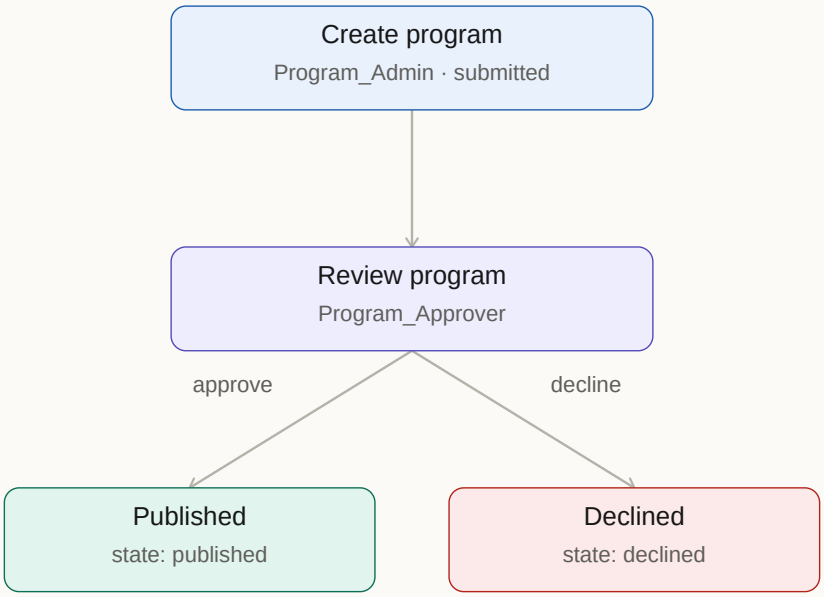

Figure 2 — Program approval flow, scoped within a single project.

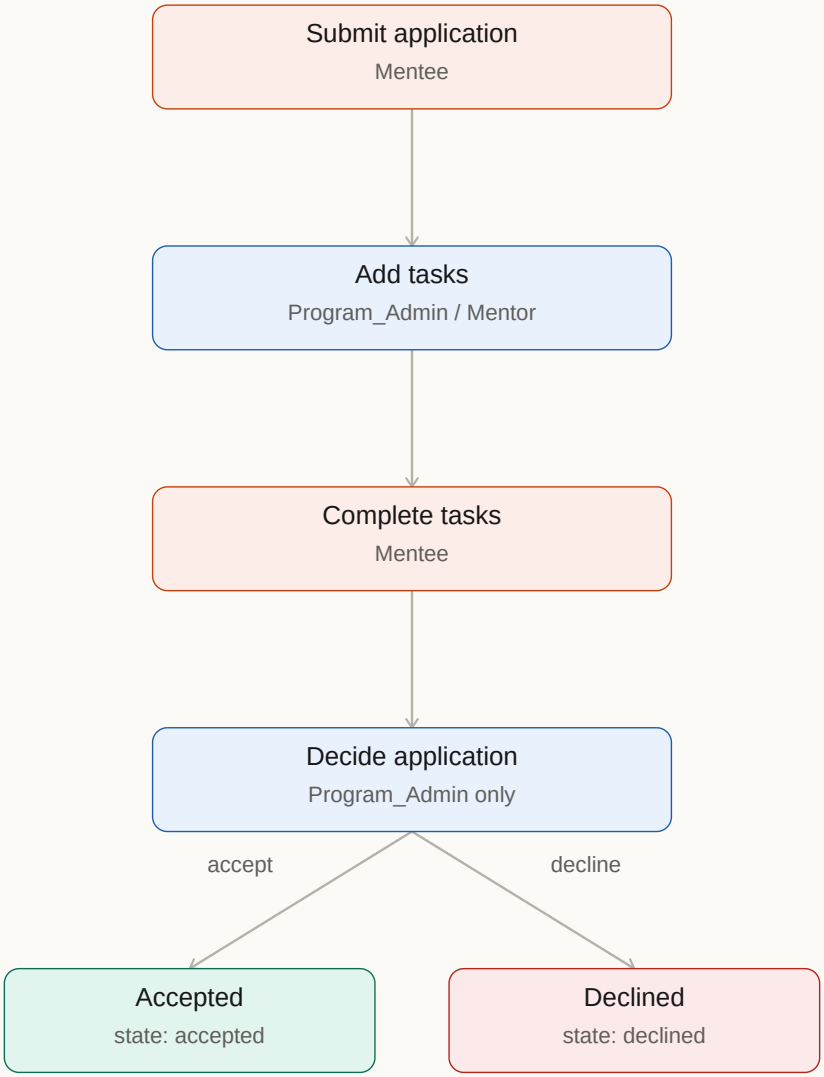

Figure 3 — Application & task flow. Program_Admin and Mentor share task authority (prerequisite and non-prerequisite alike), but only Program_Admin decides applications — reviewer-corrected; Mentor cannot accept/reject.

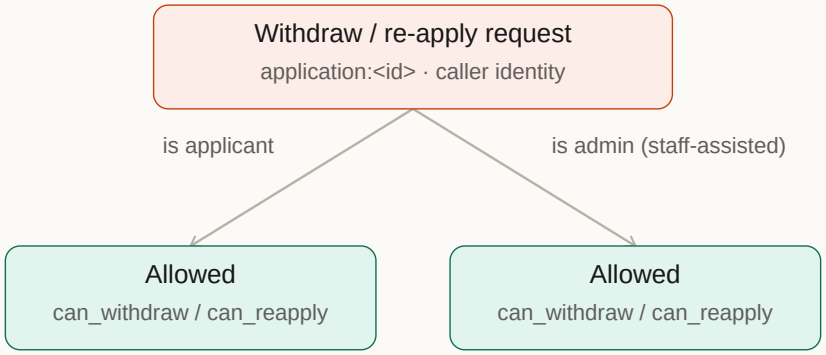

Figure 4 — Ownership check. Neither route may trust the application ID alone; the caller must resolve to `writer` (mentee or admin) before the action proceeds. Mentor alone does not qualify.

Load model and tuples

```
FGA_STORE_ID=$(fga store create --name "mentorship-fga" --api-url http://localhost:8080 \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['store']['id'])")

fga model write --store-id="$FGA_STORE_ID" --api-url http://localhost:8080 --file model.fga
fga tuple import --store-id="$FGA_STORE_ID" --api-url http://localhost:8080 --file tuples.yaml

echo "Playground: http://localhost:9000/playground"
```

Run the test suite

```
fga model test --tests tests.yaml
# Expected: Tests 15/15 passing, Checks 77/77 passing
```

Teardown

```
docker stop openfga openfga-caddy
docker rm openfga openfga-caddy
```

Integration into LFX Platform v2

The production path requires three PRs across three repos:

PR 1 — `linuxfoundation/lfx-v2-helm`

What: Add the mentorship types to the shared OpenFGA authorization model.

File: `charts/lfx-platform/files/model.fga`

Append the `project`, `program`, `application`, and `task` types from `model.fga` in this repo. The `platform` type in lfx-v2-helm can be removed or left as a stub if other services still reference it — do not let it conflict with the new `project` root.

File: `charts/lfx-platform/templates/openfga/model.yaml`

Bump the version. Adding types is a **major** bump per the file's own versioning guidelines (currently `14.3.0` → `15.0.0`):

```
- version:
    major: 15
    minor: 0
    patch: 0
```

PR title: `feat(fga): add mentorship types (project, program, application, task)`

PR 2 — `linuxfoundation/lfx-v2-fga-sync`

What: Register the mentorship service in the protected-types inventory.

File: `docs/fga-protected-types.md`

Add a row to the Services table:

```
| lfx-v2-mentorship-service | project, program, application, task | fga-contract.md |
```

Add a section under **FGA Object Types by Domain**:

```
### Mentorship: `lfx-v2-mentorship-service`

FGA contract: docs/fga-contract.md in lfx-v2-mentorship-service

| Object type   | Operations |
|---|---|
| `project`     | update_access, delete_access, member_put, member_remove |
| `program`     | update_access, delete_access, member_put, member_remove |
| `application` | update_access, delete_access, member_put, member_remove |
| `task`        | update_access, delete_access, member_put, member_remove |
```

PR title: `docs: add lfx-v2-mentorship-service to FGA protected-types inventory`

PR 3 — `linuxfoundation/lfx-v2-mentorship-service`

What: Three things — a contract doc, NATS publishers, and access check calls.

3a. Create `docs/fga-contract.md`

Document the object types, which lifecycle events map to which NATS subject, and the tuple shape for each. Use an existing service (e.g. `lfx-v2-committee-service/docs/fga-contract.md`) as a shape reference only — do not copy relations or business logic.

3b. NATS publishers

Publish to the generic `lfx.fga-sync.*` subjects consumed by `lfx-v2-fga-sync`. See docs/client-guide.md for the authoritative envelope format.

Program events:

Lifecycle event	Subject	Tuples written
Program created	`lfx.fga-sync.update_access`	`project` link, `program_admin`, initial `mentor` if any
Program deleted	`lfx.fga-sync.delete_access`	all tuples removed
Mentor invited/accepted	`lfx.fga-sync.member_put`	`mentor` relation on the program
Mentor removed	`lfx.fga-sync.member_remove`	`mentor` relation removed

Application events:

Lifecycle event	Subject	Tuples written
Application submitted	`lfx.fga-sync.update_access`	`program` link, `mentee` relation
Application withdrawn/deleted	`lfx.fga-sync.delete_access`	all tuples removed

Task events:

Lifecycle event	Subject	Tuples written
Task created	`lfx.fga-sync.update_access`	`application` link, `assignee` relation
Task deleted	`lfx.fga-sync.delete_access`	all tuples removed

3c. Replace in-service permission checks with FGA access checks

For each place the mentorship service currently enforces a permission (e.g. "is this user allowed to decide this application?"), replace it with a NATS request/reply call to `lfx.access_check.request`.

Map each existing check to the corresponding FGA relation:

Old check	FGA call
is user a program admin?	`can_invite_mentor` on `program:<id>`
can user approve a program?	`can_approve` on `program:<id>`
can user decide this application?	`can_decide` on `application:<id>`
can user update this task?	`can_update_status` on `task:<id>`
can user withdraw this application?	`can_withdraw` on `application:<id>`
can user re-apply for this application?	`can_reapply` on `application:<id>`

PR title: `feat(fga): implement FGA authorization for programs, applications, and tasks`

Test coverage

All 15 test scenarios from `tests.yaml` must pass against the merged model before the lfx-v2-helm PR is merged. After deploying to staging, replay a representative set of NATS events and verify the results against the assertions in `tests.yaml` using `lfx.access_check.request`.

Test	Key assertion
project-level program creation and approval rights	`carla` can create, `priya` can approve, `aisha` can do both
super_admin has unrestricted access to every object	`aisha` allowed on all types at all levels
program_admin has full control over their program	`carla` on `go-2026`: all ops allowed
program_admin has no access to other programs	`carla` on `rust-2026`: all denied
mentor can view, create terms, and add tasks, but cannot invite mentors or decide applications	`miguel` on `go-2026`: `can_view`, `can_view_applications`, `can_create_term`, `can_add_prerequisite_task`, `can_add_task` yes; `can_invite_mentor`, `can_decide_application` denied
mentor has zero access to programs they weren't invited to	`miguel` on `rust-2026`: all denied
mentee can view only their own application	`mia` on `app-mia`: allowed; on `app-noah`: denied
only admin decides applications — mentor cannot	`carla` on `app-mia`: `can_decide` yes; `miguel`: `can_decide` no
mentor can add both prerequisite and non-prerequisite tasks	`miguel` on `program:go-2026`: `can_add_task` yes, `can_add_prerequisite_task` yes
mentor can add tasks to applications and update task status	`miguel` on `app-mia`: `can_add_task` yes; on `task-mia-1`: `can_update_status` yes
mentee can view/update only their own tasks	`mia` on `task-mia-1`: allowed; `noah`: denied
program_admin and mentor can view and update tasks in their program	`carla` + `miguel` on `task-mia-1`: both allowed
program_approver can view and approve but not manage	`priya`: `can_view` + `can_approve` yes; ops denied
program_approver can see applications but not decide	`priya` on `app-mia`: `can_view` yes, `can_decide` no
withdraw/re-apply require ownership — applicant or admin only, never mentor alone	`mia` on `app-mia`: `can_withdraw` + `can_reapply` yes; on `app-noah`: denied; `carla` (staff-assisted) on `app-mia`: allowed; `miguel` (mentor): denied